



ATPBOOST: Learning Premise Selection in Binary Setting with ATP Feedback

Bartosz Piotrowski^{1,2}(✉) and Josef Urban¹

¹ Czech Institute of Informatics, Robotics and Cybernetics, Prague, Czech Republic

² Faculty of Mathematics, Informatics and Mechanics, University of Warsaw,
Warsaw, Poland

`bartoszpiotrowski@post.pl`

Abstract. ATPBOOST is a system for solving sets of large-theory problems by interleaving ATP runs with state-of-the-art machine learning of premise selection from the proofs. Unlike many approaches that use multi-label setting, the learning is implemented as binary classification that estimates the pairwise-relevance of (*theorem*, *premise*) pairs. ATPBOOST uses for this the fast state-of-the-art XGBoost gradient boosting algorithm. Learning in the binary setting however requires negative examples, which is nontrivial due to many alternative proofs. We discuss and implement several solutions in the context of the ATP/ML feedback loop, and show significant improvement over the multi-label approach.

1 Introduction: Machine Learning for Premise Selection

Assume that c is a conjecture which is a logical consequence of a large set of premises P . The chance of finding a proof of c by an automated theorem prover (ATP) often depends on choosing a small subset of P relevant for proving c . This is known as the *premise selection* task [1]. This task is crucial to make ATPs usable for proof automation over large formal corpora created with systems such as Mizar, Isabelle, HOL, and Coq [4]. Good methods for premise selection typically also transfer to related tasks, such as *internal proof guidance* of ATPs [8, 10, 13, 17] and *tactical guidance* of ITPs [7].

The most efficient premise selection methods use *data-driven/machine-learning* approaches. Such methods work as follows. Let T be a set of theorems with their proofs. Let C be a set of conjectures without proofs, each associated with a set of available premises that can be used to prove them. We want to learn a (statistical) model from T , which for each conjecture $c \in C$ will rank its available premises according to their relevance for producing an ATP proof of c . Two different machine learning settings can be used for this task:

1. *multilabel classification*: we treat premises used in the proofs as opaque labels and we create a model capable of labeling conjectures based on their features,

Supported by the *AI4REASON* ERC grant 649043, Czech project *AI&Reasoning* CZ.02.1.01/0.0/0.0/15_003/0000466 and European Regional Development Fund.

2. *binary classification*: here the aim of the learning model is to recognize pairwise-relevance of the (*conjecture*, *premise*) pairs, i.e. to decide what is the chance of a premise being relevant for proving the conjecture based on the features of both the conjecture and the premise.

Most of the machine learning methods for premise selection have so far used the first setting [3,9,11]. This includes fast and robust machine learning algorithms such as *naive Bayes* and *k-nearest neighbors* (k-NN) capable of multilabel classification with many examples and labels. This is needed for large formal libraries with many facts and proofs. There are however several reasons why the second approach may be better:

1. *Generality*: in binary classification it is easier to estimate the relevance of (*conjecture*, *premise*) pairs where the premise was so far unseen (i.e., not in the training data).
2. *State-of-the-art ML algorithms* are often capable of learning subtle aspects of complicated problems based on the features. The multilabel approach trades the rich feature representation of the premise for its opaque label.
3. Many state-of-the-art ML algorithms are binary classifiers or they struggle when performing multilabel classification for a large number of labels.

Recently, substantial work [2] has been done in the binary setting. In particular, applying deep learning to premise selection has improved state of the art in the field. There are however modern and efficient learning algorithms such as XGBoost [5] that are much less computationally-intensive than deep learning methods. Also, obtaining negative examples for training the binary classifiers is a very interesting problem in the context of many alternative ATP proofs and a feedback loop between the ATP and the learning system.

1.1 Premise Selection in Binary Setting with Multiple Proofs

The existence of multiple ATP proofs makes premise selection different from conventional machine learning applications. This is evident especially in the binary classification setting. The ML algorithms for recognizing pairwise relevance of (*conjecture*, *premise*) pairs require good data consisting of two (typically balanced) classes of positive and negative examples. But there is no conventional way how to construct such data in our domain. For every true conjecture there are infinitely many mathematical proofs. The ATP proofs are often based on many different sets of premises. The notions of *useful* or *superfluous premise* are only approximations of their counterparts defined for sets of premises.

As an example, consider the following frequent situation: a conjecture c can be ATP-proved with two sets of axioms: $\{p_1, p_2\}$ and $\{p_3, p_4, p_5\}$. Learning only from one of the sets as positives and presenting the other as negative (*conjecture*, *premise*) pairs may considerably distort the learned notion of a *useful premise*. This differs from the multilabel setting, where negative data are typically not used by the fast ML algorithms such as naive Bayes and k-NN. They just aggregate different positive examples into the final ranking.

Therefore, to further improve the premise selection algorithms it seems useful to consider learning from multiple proofs and to develop methods producing good negative data. The most suitable way how to do that is to allow multiple interactions of the machine learner with the ATP system. In the following section we present the ATPBOOST system, which implements several such algorithms.

2 ATPBOOST: Setting, Algorithms and Components

ATPBOOST¹ is a system for solving sets of large-theory problems by interleaving ATP runs with learning of premise selection from the proofs using the state-of-the-art XGBoost algorithm. The system implements several algorithms and consists of several components described in the following sections. Its setting is a large theory $\mathcal{T}$, extracted from a large ITP library where facts appear in a chronological order. In more detail, we assume the following inputs and notation:

1. T – names of theorems (and problems) in a large theory $\mathcal{T}$.
2. P – names of all facts (premises) in $\mathcal{T}$. We require $P \supseteq T$.
3. STATEMENTS_P of all $p \in P$ in the TPTP format [15] .
4. FEATURES_P – characterizing each $p \in P$. Here we use the same features as in [11] and write $\mathbf{f}_p$ for the (sparse) vector of features of p .
5. ORDER_P ($<_P$) – total order on P ; p may be used to prove t iff $p <_P t$. We write A_t for $\{p : p <_P t\}$, i.e. the set of premises allowed for t .
6. $\text{PROOFS}_{T'}$ for a subset $T' \subseteq T$. Each $t \in T'$ may have many proofs – denoted by $\mathcal{P}_t$. P_t denotes the premises needed for at least one proof in $\mathcal{P}_t$.

2.1 Algorithms

We first give a high-level overview and pseudocode of the algorithms implemented in ATPBOOST. Section 2.2 then describes the used components in detail.

Algorithm 1 is the simplest setting. Problems are split into the train/test sets, XGBoost learns from the training proofs, and its predictions are ATP-evaluated on the test set. This is used mainly for parameter optimization.

Algorithm 2 evaluates the trained XGBoost also on the training part, possibly finding new proofs that are used to update the training data for the next iteration. The test problems and proofs are never used for training. Negative mining may be used to find the worst misclassified premises and to correspondingly update the training data in the next iteration.

Algorithm 3 begins with no training set, starting with ATP runs on random rankings. XGBoost is trained on the ATP proofs from the previous iteration, producing new ranking for all problems for the next iteration. This is a MaLAREa-style [16] feedback loop between the ATP and the learner.

¹ The Python package is at <https://github.com/BartoszPiotrowski/ATPboost>.

2.2 Components

Below we describe the main components of the ATPBOOST algorithms and the main ideas behind them. As discussed in Sect. 1, they take into account the binary learning setting, and in particular implement the need to teach the system about multiple proofs by proper choice of examples, continuous interaction with the ATP and intelligent processing of its feedback. The components are available as procedures in our Python package.

Algorithm 1. Simple training/test split.

Require: Set of theorems T , set of premises $P \supseteq T$, PROOFS_T , FEATURES_P , STATEMENTS_P , ORDER_P , $\text{PARAMS}_{\text{set}}$, $\text{PARAMS}_{\text{model}}$.

- 1: $T_{\text{train}}, T_{\text{test}} \leftarrow \text{RANDOMLYSPLIT}(T)$
- 2: $\mathcal{D} \leftarrow \text{CREATETRAININGSET}(\text{PROOFS}_{T_{\text{train}}}, \text{FEATURES}_P, \text{ORDER}_P, \text{PARAMS}_{\text{set}})$
- 3: $\mathcal{M} \leftarrow \text{TRAINMODEL}(\mathcal{D}, \text{PARAMS}_{\text{model}})$
- 4: $\mathcal{R} \leftarrow \text{CREATERANKINGS}(T_{\text{test}}, \mathcal{M}, \text{FEATURES}_P, \text{ORDER}_P)$
- 5: $\mathcal{P} \leftarrow \text{ATPEVALUATION}(\mathcal{R}, \text{STATEMENTS}_P)$

Algorithm 2. Incremental feedback-loop with training/test split.

Require: Set of theorems T , set of premises $P \supseteq T$, FEATURES_P , STATEMENTS_P , PROOFS_T , ORDER_P , $\text{PARAMS}_{\text{set}}$, $\text{PARAMS}_{\text{model}}$, $\text{PARAMS}_{\text{negmin}}$ (optionally).

- 1: $T_{\text{train}}, T_{\text{test}} \leftarrow \text{RANDOMLYSPLIT}(T)$
- 2: $\mathcal{D} \leftarrow \text{CREATETRAININGSET}(\text{PROOFS}_{T_{\text{train}}}, \text{FEATURES}_P, \text{ORDER}_P, \text{PARAMS}_{\text{set}})$
- 3: **repeat**
- 4: $\mathcal{M} \leftarrow \text{TRAINMODEL}(\mathcal{D}, \text{PARAMS}_{\text{model}})$
- 5: $\mathcal{R}_{\text{train}} \leftarrow \text{CREATERANKINGS}(T_{\text{train}}, \mathcal{M}, \text{FEATURES}_P, \text{ORDER}_P)$
- 6: $\mathcal{R}_{\text{test}} \leftarrow \text{CREATERANKINGS}(T_{\text{test}}, \mathcal{M}, \text{FEATURES}_P, \text{ORDER}_P)$
- 7: $\mathcal{P}_{\text{train}} \leftarrow \text{ATPEVALUATION}(\mathcal{R}_{\text{train}}, \text{STATEMENTS}_P)$
- 8: $\mathcal{P}_{\text{test}} \leftarrow \text{ATPEVALUATION}(\mathcal{R}_{\text{test}}, \text{STATEMENTS}_P)$
- 9: $\text{UPDATE}(\text{PROOFS}_{\text{train}}, \mathcal{P}_{\text{train}})$
- 10: $\text{UPDATE}(\text{PROOFS}_{\text{test}}, \mathcal{P}_{\text{test}})$
- 11: **if** $\text{PARAMS}_{\text{negmin}}$ **then**
- 12: $\mathcal{D} \leftarrow \text{NEGATIVEMINING}(\mathcal{R}, \text{PROOFS}_{\text{train}}, \text{FEATURES}_P, \text{ORDER}_P, \text{PARAMS}_{\text{negmin}})$
- 13: **else**
- 14: $\mathcal{D} \leftarrow \text{CREATETRAININGSET}(\text{PROOFS}_{\text{train}}, \text{FEATURES}_P, \text{ORDER}_P, \text{PARAMS}_{\text{set}})$
- 15: **until** Number of $\text{PROOFS}_{\text{test}}$ increased after UPDATE.

Algorithm 3. Incremental feedback-loop starting with no proofs.

Require: Set of theorems T , set of premises $P \supseteq T$, FEATURES_P , STATEMENTS_P , ORDER_P , $\text{PARAMS}_{\text{set}}$, $\text{PARAMS}_{\text{model}}$, $\text{PARAMS}_{\text{negmin}}$ (optionally).

- 1: $\text{PROOFS}_T \leftarrow \emptyset$
- 2: $\mathcal{R} \leftarrow \text{CREATERANDOMRANKINGS}(T)$
- 3: $\mathcal{P} \leftarrow \text{ATPEVALUATION}(\mathcal{R}, \text{STATEMENTS}_P)$
- 4: $\text{UPDATE}(\text{PROOFS}_T, \mathcal{P})$
- 5: $\mathcal{D} \leftarrow \text{CREATETRAININGSET}(\text{PROOFS}_T, \text{FEATURES}_P, \text{ORDER}_P, \text{PARAMS}_{\text{set}})$
- 6: **repeat**
- 7: $\mathcal{M} \leftarrow \text{TRAINMODEL}(\mathcal{D}, \text{PARAMS}_{\text{model}})$
- 8: $\mathcal{R} \leftarrow \text{CREATERANKINGS}(T, \mathcal{M}, \text{FEATURES}_P, \text{ORDER}_P)$
- 9: $\mathcal{P} \leftarrow \text{ATPEVALUATION}(\mathcal{R}, \text{STATEMENTS}_P)$
- 10: $\text{UPDATE}(\text{PROOFS}_T, \mathcal{P})$
- 11: **if** $\text{PARAMS}_{\text{negmin}}$ **then**
- 12: $\mathcal{D} \leftarrow \text{NEGATIVEMINING}(\mathcal{R}, \text{PROOFS}_T, \text{FEATURES}_P, \text{ORDER}_P, \text{PARAMS}_{\text{negmin}})$
- 13: **else**
- 14: $\mathcal{D} \leftarrow \text{CREATETRAININGSET}(\text{PROOFS}_T, \text{FEATURES}_P, \text{ORDER}_P, \text{PARAMS}_{\text{set}})$
- 15: **until** Number of PROOFS_T increased after UPDATE.

CREATETRAININGSET (**PROOFS_T**, **FEATURES_P**, **ORDER_P**, **PARAMS**). This procedure constructs a **TRAININGSET** for a binary learning algorithm. This is a sparse matrix of positive/negative examples and a corresponding vector of binary labels. The examples (matrix rows) are created from **PROOFS_T** and **FEATURES_P**, respecting **ORDER_P**. Each example is a concatenation of $\mathbf{f}_t$ and $\mathbf{f}_p$, i.e., the features of a theorem t and a premise p . Positive examples express that p is relevant for proving t , whereas the negatives mean the opposite.

The default method (**SIMPLE**) creates positives from all pairs (t, p) where $p \in P_t$. Another method (**SHORT**) creates positives only from the *short* proofs of t . These are the proofs of t with at most $m + 1$ premises, where m is the minimal number of premises used in a proof from P_t . Negative examples for theorem t are chosen randomly from pairs (t, p) where $p \in A_t \setminus P_t$. The number of such randomly chosen pairs is $\text{RATIO} \cdot N_{\text{pos}}$, where N_{pos} is the number of positives and $\text{RATIO} \in \mathbb{N}$ is a parameter that needs to be optimized experimentally. Since $|A_t \setminus P_t|$ is usually much larger than $|P_t|$, it seems reasonable to have a large **RATIO**. This however increases class imbalance and the probability of presenting to the learning algorithm a *false negative*. This is a pair (t, p) where $p \notin P_t$, but there is an ATP proof of t using p that is not yet in our dataset.

TRAINMODEL (**TRAININGSET**, **PARAMS**). This procedure trains a binary learning classifier on the **TRAININGSET**, creating a **MODEL**. We use XGBoost [5] – a state-of-the-art tree-based gradient boosting algorithm performing very well in machine learning competitions. It is also much faster to train compared to deep learning methods, performs well with unbalanced training sets, and is optimized for working with sparse data. XGBoost has several important parameters, such as **NUMBEROFTREES**, **MAXDEPTH** (of trees) and **ETA** (learning rate). These parameters have significant influence on the performance and require tuning.

CREATERANKINGS (C , **MODEL**, **FEATURES_P**, **ORDER_P**). This procedure uses the trained **MODEL** to construct **RANKINGS_C** of premises from P for conjectures $c \in C \subseteq T$. Each conjecture c is paired with each premise $p <_P c$ and concatenations of $\mathbf{f}_c$ and $\mathbf{f}_p$ are passed to the **MODEL**. The **MODEL** outputs a real number in $[0, 1]$, which is interpreted as the relevance of p for proving c . The relevances are then used to sort the premises into **RANKINGS_C**.

ATPEVALUATION (**RANKINGS**, **STATEMENTS**). Any ATP can be used for evaluation. By default we use E [14]². As usual, we construct the ATP problems for several top slices (lengths 1, 2, ..., 512) of the **RANKINGS**. To remove redundant premises we *pseudo-minimize* the proofs: only the premises needed in the proofs are used as axioms and the ATP is rerun until a fixpoint is reached.

UPDATE (**OLDPROOFS**, **NEWPROOFS**). The **UPDATE** makes a union of the new and old proofs, followed by a subsumption reduction. I.e., if premises of two proofs of t are in a superset relation, the proof with the larger set is removed.

² The default time limit is 10 s and the memory limit is 2 GB. The exact default command is: `./eprover -auto-schedule -free-numbers -s -R -cpu-limit=10 -memory-limit=2000 -print-statistics -p -tstp-format problem_file`.

NEGATIVEMINING (**PROOFS_T**, **FEATURES_P**, **ORDER_P**, **PARAMS**). This is used as a more advanced alternative to **CREATETRAININGSET**. It examines the last **RANKINGS_T** for the most *misclassified positives*. I.e., for each $t \in T$ we create a set MP_t of those p that were previously ranked high for t , but no ATP proof of t was using p . We define three variants:

1. **NEGMIN_ALL**: Let m_t be the maximum rank of a t -useful premise ($p \in P_t$) in **RANKINGS_T**[t]. Then $MP_t^1 = \{p : \text{rank}_t(p) < m_t \wedge p \notin P_t\}$.
2. **NEGMIN_RANDOM**: We randomly choose into MP_t^2 only a half of MP_t^1 .
3. **NEGMIN_1**: $MP_t^3 = \{p : \text{rank}_t(p) < |P_t| \wedge p \notin P_t\}$.

The set MP_t^i is then added as negatives to the examples produced by the **CREATETRAININGSET** procedure. The idea of such negative mining is that the learner takes into account the mistakes it made in the previous iteration.

3 Evaluation

We evaluate³ the algorithms on a set of 1342 MPTP2078 [1] large (*chainy*) problems that are provable in 60 s using their small (*bushy*) versions.

Parameter Tuning: First we run Algorithm 1 to optimize the parameters. The dataset was randomly split into a train set of 1000 problems and test set of 342. For the train set, we use the proofs obtained by the 60 s run on the bushy versions. We tune the **RATIO** parameter of **CREATETRAININGSET**, and the **NUMBEROFTREES**, **MAXDEPTH** and **ETA** parameters of **TRAINMODEL**. Due to resource constraints we *a priori* assume good defaults: **RATIO** = 16, **NUMBEROFTREES** = 2000, **MAXDEPTH** = 10, **ETA** = 0.2. Then we observe how changing each parameter separately influences the results. Table 1 shows the ATP results for the **RATIO** parameter, and Fig. 1 for the model parameters.

Table 1. Influence of the **RATIO** of randomly generated negatives to positives.

RATIO	1	2	4	8	16	32	64
Proved (%)	74.0	78.4	79.0	78.7	80.1	79.8	80.1

It is clear that a high number of negatives is important. Using **RATIO** = 16 proves 6% more test problems than the balanced setting (**RATIO** = 1). It is also clear that a higher number of trees – at least 500 – improves the results. However, too many trees (over 8000) slightly decrease the performance, likely due to overfitting. The **ETA** parameter gives best results with values between 0.04 and 0.64, and the **MAXDEPTH** of trees should be around 10.

³ All the scripts we used for the evaluation are available at <https://github.com/BartoszPiotrowski/ATPboost/tree/master/experiments>.

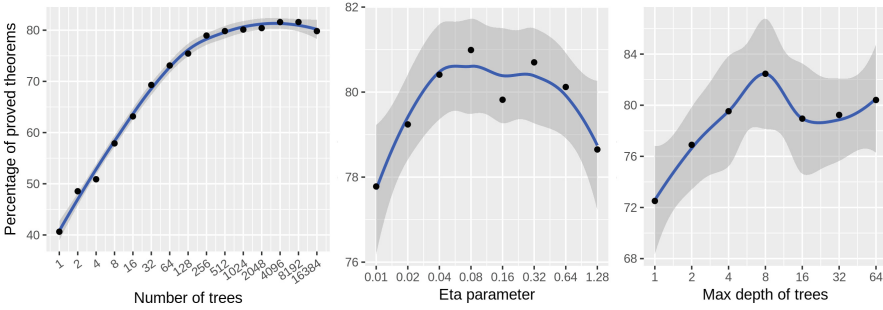


Fig. 1. ATP performance of different parameters of the XGBoost model.

We evaluate Algorithm 1 also on a much bigger ATP-provable part of MML with 29271 theorems in train part and 3253 in test. With parameters $\text{RATIO} = 20$, $\text{NUMBEROFTREES} = 4000$, $\text{MAXDEPTH} = 10$ and $\text{ETA} = 0.2$ we proved 58.78% theorems (1912). This is a 15.7% improvement over k-NN, which proved 50.81% (1653) theorems. For a comparison, the improvement over k-NN obtained (with much higher ATP time limits) with deep learning in [2] was 4.3%.

Incremental Feedback Loop with Train/Test Split: This experiment evaluates Algorithm 2, testing different methods of negative mining. The train/test split and the values of the parameters RATIO , NUMBEROFTREES , MAXDEPTH , ETA are taken from the previous experiment. We test six methods in parallel. Two XGB methods (`SIMPLE` and `SHORT`) are the variants of the `CREATETRAININGSET` procedure, three XGB methods (`NEGMIN_ALL`, `NEGMIN_RAND` and `NEGMIN_1`) are the variants of `NEGATIVEMINING`, and the last one is a k-NN learner similar to the one from [11], used here for comparison. The experiment starts with the same proofs for training theorems as in the previous one, and we performed 30 rounds of the feedback loop. Figure 2 shows the results.

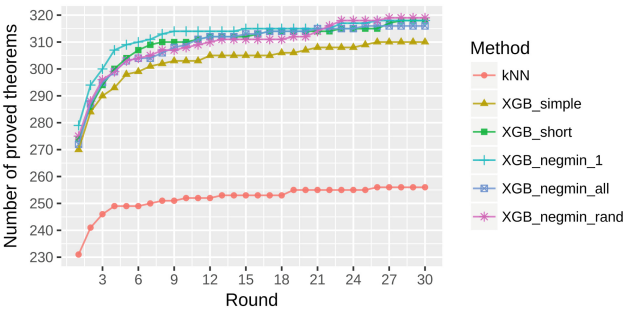


Fig. 2. Number of proved theorems in subsequent iterations of Algorithm 2.

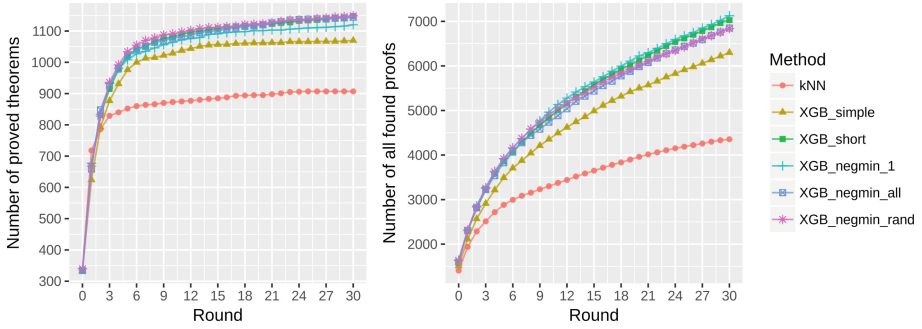


Fig. 3. Number of proved theorems (left) and number of all found proofs (right) in subsequent rounds of the experiment corresponding to Algorithm 3.

All the new methods largely outperform k-NN, and XGB_SHORT is much better than XGB_SIMPLE. I.e., positives from too many proofs seem harmful, as in [12] where this was observed with k-NN. The differences between the XGB variants SHORT, NEGMIN_1, NEGMIN_ALL, and NEGMIN_RAND do not seem significant and all perform well. At the end of the loop (30th round) 315–319 theorems of the 342 (ca 93%) are proved.

Incremental Feedback-loop with no Initial Proofs: The final experiment corresponds to Algorithm 3. There is no train/test split and no initial proofs. The first ATP evaluation is done on random rankings, proving 335 theorems out of the 1342. Then the loop starts running with the same options as in the previous experiment. Figure 3 shows the numbers of theorems that were proved in the subsequent rounds, as well as the growth of the total number of different proofs. This is important, because all these proofs are taken into account by the machine learning. Again, k-NN is the weakest and XGB_SIMPLE is worse than the rest of the methods, which are statistically indistinguishable. In the last round XGB_NEGMIN_RAND proves 1150 (86%) theorems. This is a 26.8% improvement over k-NN (907) and 7.7% more than XGB_SIMPLE (1068).

References

1. Alama, J., Heskes, T., Kühlwein, D., Tsivtsivadze, E., Urban, J.: Premise selection for mathematics by corpus analysis and kernel methods. *J. Autom. Reason.* **52**(2), 191–213 (2014)
2. Alemi, A.A., Chollet, F., Irving, G., Szegedy, C., Urban, J. (eds.): DeepMath - deep sequence models for premise selection (2016). <http://dblp.uni-trier.de/rec/bibtex/conf/nips/IrvingSAECU16>
3. Blanchette, J.C., Greenaway, D., Kaliszyk, C., Kühlwein, D., Urban, J.: A learning-based fact selector for Isabelle/HOL. *J. Autom. Reason.* **57**(3), 219–244 (2016)
4. Blanchette, J.C., Kaliszyk, C., Paulson, L.C., Urban, J.: Hammering towards QED. *J. Formal. Reason.* **9**(1), 101–148 (2016)

5. Chen, T., Guestrin, C.: XGBoost: a scalable tree boosting system (2016). <http://dblp.uni-trier.de/rec/bibtex/conf/kdd/ChenG16>
6. Eiter, T., Sands, D. (eds.) LPAR-21, 21st International Conference on Logic for Programming, Artificial Intelligence and Reasoning, Maun, Botswana, 7–12 May 2017. EPiC Series in Computing, vol. 46. EasyChair (2017)
7. Gauthier, T., Kaliszyk, C., Urban, J.: TacticToe: learning to reason with HOL4 tactics. In: Eiter and Sands [6], pp. 125–143 (2017)
8. Jakubův, J., Urban, J.: ENIGMA: efficient learning-based inference guiding machine. In: Geuvers, H., England, M., Hasan, O., Rabe, F., Teschke, O. (eds.) CICM 2017. LNCS (LNAI), vol. 10383, pp. 292–302. Springer, Cham (2017). https://doi.org/10.1007/978-3-319-62075-6_20
9. Kaliszyk, C., Urban, J.: Learning-assisted automated reasoning with flyspeck. *J. Autom. Reason.* **53**(2), 173–213 (2014)
10. Kaliszyk, C., Urban, J.: FEMaLeCoP: fairly efficient machine learning connection prover. In: Davis, M., Fehnker, A., McIver, A., Voronkov, A. (eds.) LPAR 2015. LNCS, vol. 9450, pp. 88–96. Springer, Heidelberg (2015). https://doi.org/10.1007/978-3-662-48899-7_7
11. Kaliszyk, C., Urban, J.: MizAR 40 for Mizar 40. *J. Autom. Reason.* **55**(3), 245–256 (2015)
12. Kuehlwein, D., Urban, J.: Learning from multiple proofs: first experiments. In: Fontaine, P., Schmidt, R.A., Schulz, S. (eds.) PAAR-2012, EPiC Series, vol. 21, pp. 82–94. EasyChair (2013)
13. Loos, S.M., Irving, G., Szegedy, C., Kaliszyk, C.: Deep network guided proof search. In: Eiter and Sands [6], pp. 85–105 (2017)
14. Schulz, S.: E - a brainiac theorem prover. *AI Commun.* **15**(2–3), 111–126 (2002)
15. Sutcliffe, G.: The TPTP problem library and associated infrastructure. *J. Autom. Reason.* **43**(4), 337–362 (2009)
16. Urban, J., Sutcliffe, G., Pudlák, P., Vyskočil, J.: MaLAREa SG1 - machine learner for automated reasoning with semantic guidance. In: Armando, A., Baumgartner, P., Dowek, G. (eds.) IJCAR 2008. LNCS (LNAI), vol. 5195, pp. 441–456. Springer, Heidelberg (2008). https://doi.org/10.1007/978-3-540-71070-7_37
17. Urban, J., Vyskočil, J., Štěpánek, P.: MaLeCoP machine learning connection prover. In: Brünner, K., Metcalfe, G. (eds.) TABLEAUX 2011. LNCS (LNAI), vol. 6793, pp. 263–277. Springer, Heidelberg (2011). https://doi.org/10.1007/978-3-642-22119-4_21